Java Test Paper

1. Write a console-based menu driven PhoneBook application to manage user contacts. Following operations are expected:
   1. Add new Contact
   2. Display All Contact Details
   3. Update Contact details for given Name & Date of Birth
   4. Remove all Contacts who are above 80 years

   You can define a class Contact with the following fields -
   a. Phone Number (Required, Updatable)
   b. Name (Required, Non-Updatable)
   c. Date of Birth (Required, Non-Updatable)
   d. Email (Optional, Updatable)
   Notes/Constraints: Use appropriate java collection to store contacts in memory.
   A phone number belongs to only one person.
   A person can have multiple phone numbers.
   Name & Date of Birth are not updatable once entered into the system.
   Two persons cannot have same Name & Date of Birth.

   Follow Java conventions when naming packages, classes, methods and variables.

   Marks Distribution:
   Basic structure of program including class Contact & validations [10 Marks]
   Implementation of functionality 1 [6 Marks]
   Implementation of functionality 2 [5 Marks]
   Implementation of functionality 3 [6 Marks]
   Implementation of functionality 4 [5 Marks]
   Java naming convention, exception handling [8 Marks]

1. **Contact Class**

```
import java.time.LocalDate;

public class Contact {
    private String phoneNumber;
    private final String name;          // Non-updatable
    private final LocalDate dateOfBirth; // Non-updatable
    private String email;               // Optional, Updatable

    // Constructor with validation
    public Contact(String phoneNumber, String name, LocalDate dateOfBirth, String email) {
        validatePhoneNumber(phoneNumber);
        this.phoneNumber = phoneNumber;
        this.name = name;
        this.dateOfBirth = dateOfBirth;
        this.email = email;
    }

    // Getters
    public String getPhoneNumber() { return phoneNumber; }
    public String getName() { return name; }
    public LocalDate getDateOfBirth() { return dateOfBirth; }
    public String getEmail() { return email; }

    // Setters for updatable fields
    public void setPhoneNumber(String phoneNumber) {
        validatePhoneNumber(phoneNumber);
        this.phoneNumber = phoneNumber;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    // Validation for phone number (basic)
    private void validatePhoneNumber(String phoneNumber) {
        if (phoneNumber == null || phoneNumber.length() != 10 || !phoneNumber.matches("\\d+")) {
            throw new IllegalArgumentException("Invalid phone number. It must be 10 digits.");
        }
    }

    @Override
    public String toString() {
        return "Name: " + name + ", Phone: " + phoneNumber + ", DOB: " + dateOfBirth + ", Email: " + (email != null ?
email : "N/A");
    }
}
```

2. PhoneBook Class

```
import java.time.LocalDate;
import java.time.Period;
import java.util.*;
```

```
public class PhoneBook {
    private final Map<String, Contact> contacts = new HashMap<>(); // Name + DOB as unique key

    // Add new contact
    public void addContact(Contact contact) {
        String key = generateKey(contact.getName(), contact.getDateOfBirth());
        if (contacts.containsKey(key)) {
            throw new IllegalArgumentException("Contact with the same Name and Date of Birth already exists.");
        }
        contacts.put(key, contact);
    }

    // Display all contacts
    public void displayAllContacts() {
        if (contacts.isEmpty()) {
            System.out.println("No contacts available.");
        } else {
            contacts.values().forEach(System.out::println);
        }
    }

    // Update contact details
    public void updateContact(String name, LocalDate dob, String newPhoneNumber, String newEmail) {
        String key = generateKey(name, dob);
        Contact contact = contacts.get(key);
        if (contact != null) {
            if (newPhoneNumber != null) {
                contact.setPhoneNumber(newPhoneNumber);
            }
            if (newEmail != null) {
                contact.setEmail(newEmail);
            }
            System.out.println("Contact updated successfully.");
        } else {
            System.out.println("Contact not found.");
        }
    }

    // Remove contacts above 80 years of age
    public void removeOldContacts() {
        Iterator<Map.Entry<String, Contact>> iterator = contacts.entrySet().iterator();
        while (iterator.hasNext()) {
            Map.Entry<String, Contact> entry = iterator.next();
            Contact contact = entry.getValue();
            if (calculateAge(contact.getDateOfBirth()) > 80) {
                iterator.remove();
            }
        }
        System.out.println("Contacts above 80 years have been removed.");
    }

    // Helper to calculate age from DOB
    private int calculateAge(LocalDate dob) {
        return Period.between(dob, LocalDate.now()).getYears();
    }

    // Helper to generate unique key
    private String generateKey(String name, LocalDate dob) {
        return name + "_" + dob.toString();
    }
}
```

3. Main Application (PhoneBookApp)

```
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.util.Scanner;

public class PhoneBookApp {

    private static final PhoneBook phoneBook = new PhoneBook();
    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        boolean exit = false;
        while (!exit) {
            displayMenu();
            int choice = getChoice();
            switch (choice) {
                case 1 -> addNewContact();
                case 2 -> phoneBook.displayAllContacts();
                case 3 -> updateContact();
                case 4 -> phoneBook.removeOldContacts();
                case 5 -> exit = true;
                default -> System.out.println("Invalid choice. Please try again.");
            }
        }
    }

    private static void displayMenu() {
        System.out.println("\nPhoneBook Menu:");
        System.out.println("1. Add New Contact");
        System.out.println("2. Display All Contact Details");
        System.out.println("3. Update Contact Details");
        System.out.println("4. Remove Contacts Above 80 Years");
        System.out.println("5. Exit");
    }

    private static int getChoice() {
        System.out.print("Enter your choice: ");
        return scanner.nextInt();
    }

    private static void addNewContact() {
        try {
            System.out.print("Enter Name: ");
            String name = scanner.next();

            System.out.print("Enter Date of Birth (YYYY-MM-DD): ");
            LocalDate dob = LocalDate.parse(scanner.next());

            System.out.print("Enter Phone Number: ");
            String phone = scanner.next();

            System.out.print("Enter Email (optional): ");
            String email = scanner.next();

            Contact contact = new Contact(phone, name, dob, email);
            phoneBook.addContact(contact);
            System.out.println("Contact added successfully.");
        } catch (IllegalArgumentException | DateTimeParseException e) {
            System.out.println("Error: " + e.getMessage());
```

```
        }
    }

    private static void updateContact() {
        try {
            System.out.print("Enter Name: ");
            String name = scanner.next();

            System.out.print("Enter Date of Birth (YYYY-MM-DD): ");
            LocalDate dob = LocalDate.parse(scanner.next());

            System.out.print("Enter new Phone Number (or leave blank): ");
            String phone = scanner.next();

            System.out.print("Enter new Email (or leave blank): ");
            String email = scanner.next();

            phoneBook.updateContact(name, dob, phone.isEmpty() ? null : phone, email.isEmpty() ? null : email);
        } catch (IllegalArgumentException | DateTimeParseException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

2. **A company Style With Pen sells stylish Pens online.**
   Company needs an interface for site admin to manage stocks.

   Q. 1) Write a console-based menu driven java program for site ADMIN to perform following operations: (40 Marks] tip)

   1. Add new Pen 15 Marks]
   2. Update stock of a Pen 15 Marks]
   3. Set discount of 20% for all the pens which are not at all sold in last 3 months [10 Marks]
   4. Remove Pens which are never sold once listed in 9 months [10 Marks]

   You can define a class Pen with the following fields-
   a. ID (unique identifier for each Pen, should be generated automatically)
   b. Brand (Example: Cello, Parker, Reynolds etc.)
   c. Color
   d. Ink Color
   e. Material (Example: Plastic, Alloy Steel, Metal etc.)
   f. Stock (Available quantity)
   g.Stock Update Date (it changed every time when admin update stock or user order executed)
   h. Stock Listing Date (date on which product is added to site for sale)
   i. Price (in INR) - Discounts (in percentage)

Note:
You can use java collection to store items in memory. Storing data in relational database in not expected.

1.Basic structure of program including class Pen, Enums, collections [5 Marks]
2.Java naming convention, exception handling [5 Marks]

Solution:

Directory Structure:

```
src
├── app
│   └── PenInventoryApp.java
├── model
│   └── Pen.java
└── service
    └── PenInventory.java
```

1. model.Pen Class (Package: `model`)

```
package model;

import java.time.LocalDate;

public class Pen {
    private static int idCounter = 1; // Auto-generated unique ID for each pen
    private final int id;
    private String brand;
    private String color;
    private String inkColor;
    private String material;
    private int stock;
    private LocalDate stockUpdateDate;
    private final LocalDate stockListingDate;
    private double price;
    private double discount;

    public Pen(String brand, String color, String inkColor, String material, int stock, double price) {
        this.id = idCounter++;
        this.brand = brand;
        this.color = color;
        this.inkColor = inkColor;
        this.material = material;
        this.stock = stock;
        this.stockUpdateDate = LocalDate.now();
        this.stockListingDate = LocalDate.now();
        this.price = price;
        this.discount = 0.0;
    }

    // Getters
    public int getId() { return id; }
```

```
    public String getBrand() { return brand; }
    public String getColor() { return color; }
    public String getInkColor() { return inkColor; }
    public String getMaterial() { return material; }
    public int getStock() { return stock; }
    public LocalDate getStockUpdateDate() { return stockUpdateDate; }
    public LocalDate getStockListingDate() { return stockListingDate; }
    public double getPrice() { return price; }
    public double getDiscount() { return discount; }

    // Setters for updatable fields
    public void setStock(int stock) {
        this.stock = stock;
        this.stockUpdateDate = LocalDate.now();
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public void setDiscount(double discount) {
        this.discount = discount;
    }

    @Override
    public String toString() {
        return "Pen{" +
                "ID=" + id +
                ", Brand='" + brand + '\'' +
                ", Color='" + color + '\'' +
                ", Ink Color='" + inkColor + '\'' +
                ", Material='" + material + '\'' +
                ", Stock=" + stock +
                ", Stock Update Date=" + stockUpdateDate +
                ", Stock Listing Date=" + stockListingDate +
                ", Price=" + price +
                ", Discount=" + discount + "%" +
                '}';
    }
}
```

2. service.PenInventory Class (Package: `service`)

```
package service;

import model.Pen;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.*;

public class PenInventory {
    private final Map<Integer, Pen> inventory = new HashMap<>();

    // Add a new Pen to the inventory
    public void addPen(Pen pen) {
        inventory.put(pen.getId(), pen);
        System.out.println("Pen added successfully: " + pen);
    }
```

```
    // Update the stock of a Pen
    public void updateStock(int penId, int newStock) {
        Pen pen = inventory.get(penId);
        if (pen != null) {
            pen.setStock(newStock);
            System.out.println("Stock updated successfully for Pen ID: " + penId);
        } else {
            System.out.println("Pen not found with ID: " + penId);
        }
    }

    // Set discount for pens not sold in the last 3 months
    public void setDiscountForUnsoldPens() {
        for (Pen pen : inventory.values()) {
            long monthsSinceLastUpdate = ChronoUnit.MONTHS.between(pen.getStockUpdateDate(), LocalDate.now());
            if (monthsSinceLastUpdate >= 3) {
                pen.setDiscount(20.0);
            }
        }
        System.out.println("Discount applied to unsold pens.");
    }

    // Remove pens that were never sold in the last 9 months
    public void removeUnsoldPens() {
        Iterator<Pen> iterator = inventory.values().iterator();
        while (iterator.hasNext()) {
            Pen pen = iterator.next();
            long monthsSinceListing = ChronoUnit.MONTHS.between(pen.getStockListingDate(), LocalDate.now());
            if (monthsSinceListing >= 9 && pen.getStock() > 0) {
                iterator.remove();
            }
        }
        System.out.println("Pens never sold in 9 months have been removed.");
    }

    // Display all pens
    public void displayAllPens() {
        if (inventory.isEmpty()) {
            System.out.println("No pens available in the inventory.");
        } else {
            inventory.values().forEach(System.out::println);
        }
    }
}
```

3. app.PenInventoryApp Class (Package: app)

```
package app;

import model.Pen;
import service.PenInventory;

import java.util.Scanner;

public class PenInventoryApp {
    private static final PenInventory penInventory = new PenInventory();
    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        boolean exit = false;
```

```
        while (!exit) {
            displayMenu();
            int choice = getChoice();
            switch (choice) {
                case 1 -> addNewPen();
                case 2 -> updateStock();
                case 3 -> applyDiscountToUnsoldPens();
                case 4 -> removeUnsoldPens();
                case 5 -> displayAllPens();
                case 6 -> exit = true;
                default -> System.out.println("Invalid choice. Please try again.");
            }
        }
    }

    private static void displayMenu() {
        System.out.println("\nPen Inventory Admin Menu:");
        System.out.println("1. Add New Pen");
        System.out.println("2. Update Stock of a Pen");
        System.out.println("3. Apply Discount for Unsold Pens");
        System.out.println("4. Remove Unsold Pens");
        System.out.println("5. Display All Pens");
        System.out.println("6. Exit");
    }

    private static int getChoice() {
        System.out.print("Enter your choice: ");
        return scanner.nextInt();
    }

    private static void addNewPen() {
        System.out.print("Enter Brand: ");
        String brand = scanner.next();

        System.out.print("Enter Color: ");
        String color = scanner.next();

        System.out.print("Enter Ink Color: ");
        String inkColor = scanner.next();

        System.out.print("Enter Material: ");
        String material = scanner.next();

        System.out.print("Enter Stock: ");
        int stock = scanner.nextInt();

        System.out.print("Enter Price: ");
        double price = scanner.nextDouble();

        Pen pen = new Pen(brand, color, inkColor, material, stock, price);
        penInventory.addPen(pen);
    }

    private static void updateStock() {
        System.out.print("Enter Pen ID: ");
        int penId = scanner.nextInt();

        System.out.print("Enter New Stock Quantity: ");
        int newStock = scanner.nextInt();

        penInventory.updateStock(penId, newStock);
```

```
    }

    private static void applyDiscountToUnsoldPens() {
        penInventory.setDiscountForUnsoldPens();
    }

    private static void removeUnsoldPens() {
        penInventory.removeUnsoldPens();
    }

    private static void displayAllPens() {
        penInventory.displayAllPens();
    }
}
```

3. **Q1. Develop a java program to manage a list of grocery items. Each grocery item should have the following attributes: [40 Marks]**
   Name
   Price per unit
   Quantity in stock
   Stock update date and time ((it changed every time when quantity changes) [5Marks]

   Your program should provide the following functionalities:
   1. Add a new grocery item to the list. [5 Marks]
   2. Update the quantity of a grocery item in stock. [5 Marks]
   3. Display the list of grocery items including their name, prices and quantities. [5Marks]
   4. Remove all empty stock items. (quantity in stock is zero). [10 Marks]
   5. Display all products for which stock updated (quantity changed) in last 3 days. [10 Marks)

   Note: You can use java collection to store items in memory. Storing data in relational database in not expected.

   Use - New Floder file

4. **Develop a console-based menu driven java program to manage grocery items. Each grocery item should have the Name Price per unit Quantity in stock Stock update date and time (it changed every time when quantity changes)**
5.
6. **Your program should provide the following functionalities:**
7. **1. Add a new grocery item to the list.**
8. **2. Update the quantity of a grocery item in stock.**
9. **3. Display the list of grocery items including their name, price and quantities,**
10. **4. Remove all items which are out of stock.**
11. **5. Display the first 10 items with the lowest quantity in stock.**

12.
13. Note: Use appropriate java collection to store items in memory.
14. Follow Java conventions when naming packages, classes, methods and variables.
15.
16. Marks Distribution:
17. Basic structure of program including validations to ensure that negative prices & quantities are n Implementation of functionalay 1 [5 Marks]
18. Implementation of functionality 2 15 Marka)
19. Implementation of functionality 3 15 Marka)
20. Implementation of functionality 4 15 Marks)
21. Implementation of functionality 5 15 Marks) Java naming conventions, exceptions handling (5 Marks)

Directory Structure:

css
Copy code

```
src
├── app
│   └── GroceryApp.java
├── model
│   └── GroceryItem.java
└── service
    └── GroceryService.java
```

1. **model.GroceryItem Class (Package: `model`)**

```
package model;

import java.time.LocalDateTime;

public class GroceryItem {
    private String name;
    private double pricePerUnit;
    private int quantity;
    private LocalDateTime stockUpdateDateTime;

    public GroceryItem(String name, double pricePerUnit, int quantity) {
        this.name = name;
        this.pricePerUnit = pricePerUnit;
        this.quantity = quantity;
        this.stockUpdateDateTime = LocalDateTime.now();
    }

    // Getters and Setters
    public String getName() {
        return name;
    }
```

```
    public double getPricePerUnit() {
        return pricePerUnit;
    }

    public void setPricePerUnit(double pricePerUnit) {
        if (pricePerUnit >= 0) {
            this.pricePerUnit = pricePerUnit;
        } else {
            System.out.println("Price cannot be negative.");
        }
    }

    public int getQuantity() {
        return quantity;
    }

    public void updateQuantity(int newQuantity) {
        if (newQuantity >= 0) {
            this.quantity = newQuantity;
            this.stockUpdateDateTime = LocalDateTime.now();  // Update stock date when quantity changes
        } else {
            System.out.println("Quantity cannot be negative.");
        }
    }

    public LocalDateTime getStockUpdateDateTime() {
        return stockUpdateDateTime;
    }

    @Override
    public String toString() {
        return "GroceryItem{" +
                "Name='" + name + '\'' +
                ", Price=" + pricePerUnit +
                ", Quantity=" + quantity +
                ", Stock Update Date=" + stockUpdateDateTime +
                '}';
    }
}
```

2. service.GroceryService Class (Package: `service`)

```
package service;

import model.GroceryItem;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class GroceryService {
    private final List<GroceryItem> groceryItems = new ArrayList<>();

    // Add a new grocery item
    public void addGroceryItem(GroceryItem item) {
        groceryItems.add(item);
        System.out.println("Added: " + item);
    }

    // Update the quantity of an existing grocery item
    public void updateQuantity(String name, int newQuantity) {
```

```
        for (GroceryItem item : groceryItems) {
            if (item.getName().equalsIgnoreCase(name)) {
                item.updateQuantity(newQuantity);
                System.out.println("Updated quantity for: " + name);
                return;
            }
        }
        System.out.println("Item not found: " + name);
    }

    // Display all grocery items
    public void displayAllItems() {
        if (groceryItems.isEmpty()) {
            System.out.println("No grocery items available.");
        } else {
            groceryItems.forEach(System.out::println);
        }
    }

    // Remove items that are out of stock
    public void removeOutOfStockItems() {
        groceryItems.removeIf(item -> item.getQuantity() == 0);
        System.out.println("Removed all out-of-stock items.");
    }

    // Display the first 10 items with the lowest quantity in stock
    public void displayLowestStockItems() {
        groceryItems.stream()
                .sorted(Comparator.comparingInt(GroceryItem::getQuantity))
                .limit(10)
                .forEach(System.out::println);
    }
}
```

3. app.GroceryApp Class (Package: app)

```
package app;

import model.GroceryItem;
import service.GroceryService;

import java.util.Scanner;

public class GroceryApp {
    private static final GroceryService groceryService = new GroceryService();
    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        boolean exit = false;
        while (!exit) {
            displayMenu();
            int choice = getChoice();
            switch (choice) {
                case 1 -> addNewItem();
                case 2 -> updateItemQuantity();
                case 3 -> groceryService.displayAllItems();
                case 4 -> groceryService.removeOutOfStockItems();
                case 5 -> groceryService.displayLowestStockItems();
                case 6 -> exit = true;
                default -> System.out.println("Invalid choice. Please try again.");
```

```
            }
        }
    }

    private static void displayMenu() {
        System.out.println("\nGrocery Management Menu:");
        System.out.println("1. Add New Grocery Item");
        System.out.println("2. Update Quantity of a Grocery Item");
        System.out.println("3. Display All Grocery Items");
        System.out.println("4. Remove Out of Stock Items");
        System.out.println("5. Display First 10 Items with Lowest Quantity");
        System.out.println("6. Exit");
    }

    private static int getChoice() {
        System.out.print("Enter your choice: ");
        return scanner.nextInt();
    }

    private static void addNewItem() {
        System.out.print("Enter item name: ");
        String name = scanner.next();

        System.out.print("Enter price per unit: ");
        double price = scanner.nextDouble();
        if (price < 0) {
            System.out.println("Price cannot be negative.");
            return;
        }

        System.out.print("Enter quantity: ");
        int quantity = scanner.nextInt();
        if (quantity < 0) {
            System.out.println("Quantity cannot be negative.");
            return;
        }

        GroceryItem item = new GroceryItem(name, price, quantity);
        groceryService.addGroceryItem(item);
    }

    private static void updateItemQuantity() {
        System.out.print("Enter item name: ");
        String name = scanner.next();

        System.out.print("Enter new quantity: ");
        int quantity = scanner.nextInt();
        if (quantity < 0) {
            System.out.println("Quantity cannot be negative.");
            return;
        }

        groceryService.updateQuantity(name, quantity);
    }
}
```